Министерство науки и высшего образования Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа прикладной математики и информатики
Кафедра системного программирования ИСП РАН
Лаборатория (laboratory name)

Выпускная квалификационная работа бакалавра

Разработка системы логгирования из гостевой системы напрямую в файл на хосте на основе MPSC кольцевого буфера

Автор:

Студент Б05-232 группы

Абубакиров Владимир Андреевич

Научный руководитель:

Кандидат физико-математических наук

Камкин Александр Сергеевич

Научный консультант:

научная степень

Скородумов Дмитрий Павлович

Москва 2026

Аннотация

Разработка системы логгирования из гостевой системы напрямую в файл на хосте на основе MPSC кольцевого буфера

Абубакиров Владимир Андреевич

Современные платформы виртуализации реализуют сложный набор подсистем, обеспечивающих взаимодействие между гостевыми и хостовыми операционными системами. Однако для задач низкоуровневой отладки, профилирования и формирования корректных отчётов об ошибках критически важным является наличие высокоэффективного канала передачи диагностических данных из гостевой операционной системы на хост. В рамках работы был разработан подобный механизм, основанный на lock-free кольцевом буфере в общей памяти.

Упомянуть про сравнение с virtio-serial / virtio-console

Содержание

1 Введение **4**

2 Архитектура виртуальной машины stratovirt **10**

3 Драйверы в среде UEFI **11**

4 Драйверы в операционной системе Windows **12**

5 Реализация механизма сбора диагностической информации **13**

6 Заключение **15**

1 Введение

Развитие информационной инфраструктуры, начиная от облачных платформ и центров обработки данных, заканчивая персональными компьютерами, всё чаще полагается на технологии виртуализации.

Виртуализация — группа технологий, основанных на преобразовании формата или параметров программных или сетевых запросов к компьютерным ресурсам с целью обеспечения независимости процессов обработки информации от программной или аппаратной платформы информационной системы. [1]

Простые пользователи используют т.н. виртуальные машины для запуска альтернативных операционных систем на своих рабочих станциях для тестирования подозрительных программ, для разработки и обучения. Облачные провайдеры используют виртуальные машины для обеспечения работы серверных платформ: в частности, для предоставления пользователям возможности аренды вычислительных ресурсов. Последним это позволяет не покупать собственное оборудование для разворачивания инфраструктуры, масштабировать мощности под нагрузку и платить только за фактическое использование ресурсов.

Виртуальная машина — виртуальная вычислительная система, которая состоит из виртуальных устройств обработки, хранения и передачи данных и которая дополнительно может содержать программное обеспечение и пользовательские данные. [2]

Отдельным фактором, влияющим на рост спроса на технологии виртуализации, является развитие машинного обучения и искусственного интеллекта. Развертывание и обучение моделей требует больших вычислительных ресурсов (мощных GPU и NPU). Облачные провайдеры предоставляют виртуальные машины с таким оборудованием, позволяя бизнесу и частным поль-

[1] ГОСТ Р 56938-2016. Национальный стандарт Российской Федерации. Защита информации. Защита информации при использовании технологий виртуализации. Общие положения

[2] ГОСТ Р 56938-2016. Национальный стандарт Российской Федерации. Защита информации. Защита информации при использовании технологий виртуализации. Общие положения

зователям проводить исследования, не приобретая дорогостоящее оборудование. Это дополнительно увеличивает потребность эффективного использования аппаратного обеспечения, что также достигается при помощи технологий виртуализации.

Развитие технологий виртуализации предъявляет повышенные требования к инструментам диагностики и отладки, которые должны обеспечивать сбор данных из гостевых операционных систем с минимальным влиянием на их временные характеристики. В задачах логгирования и профилирования ключевыми являются низкая задержка и высокая пропускная способность передачи диагностической информации. В связи с этим возникает необходимость создания специализированного канала связи «гость – хост», оптимизированного для однонаправленной потоковой передачи данных.

Ситуация усугубляется тем, что на одной физической рабочей станции может одновременно функционировать несколько виртуальных машин. Каждая из них генерирует свои собственные логи, операции записи которых конкурируют за общие аппаратные ресурсы. Таким образом, механизм сбора диагностических данных должен работать с минимальной задержкой не только для отдельно взятой виртуальной машины, но и масштабироваться при одновременной работе множества экземпляров.

Ещё одной особенностью современных виртуальных машин является наличие сопутствующего программного обеспечения, позволяющего пользователям бесшовно с ними взаимодействовать. Например, инструменты, обеспечивающие общий буфер обмена, перенаправление USB-устройств, синхронизацию времени и разрешения экрана. Эти системы в процессе своей работы также генерируют диагностические события, которые могут быть полезны для выявления ошибок у пользователей. Таким образом, система сбора диагностической информации должна эффективно работать с учётом функционирования нескольких самостоятельных компонентов внутри виртуальной машины.

Разработка подобной системы передачи диагностических данных под-

разумевает создание драйвера в гостевой операционной системе, который взаимодействует со специальным виртуальным устройством на стороне эмулятора. Данное устройство выступает в роле конечного приёмника сообщений, обеспечивая их сбор и последующую печать в файл. Такой подход позволяет сохранить прозрачность для пользователя, так как гостевые операционные системы воспринимают устройство как стандартное аппаратное обеспечение.

Основой данного механизма является общая память, разделяемая между гостевой операционной системой и гипервизором. Такой подход позволяет организовать канал передачи данных без применения медленных операций ввода-вывода и без дополнительных копирований между буферами.

Сложность разработки заключается в том, что в угоду минимальных задержек, синхронизация доступа к буферу осуществляется без блокировок, благодаря использованию атомарных операций, что исключает риски взаимных блокировок и обеспечивает более детерминированное время выполнения операций. Такой подход позволяет достичь высокой пропускной способности и минимальной задержки при операциях записи, что особенно важно для работы с несколькими виртуальными машинами.

Немаловажной задачей также является реализовать механизм передачи сообщений из среды BIOS (UEFI). На этом этапе происходит первичная инициализация аппаратный компонентов и сбои приводят невозможности дальнейшей загрузки системы. Отсутствие средств сбора отладочной информации существенно затрудняет выявление причин отказов.

BIOS представляет собой интерфейс между программным обеспечением и аппаратными средствами, который позволяет им общаться и взаимодействовать друг с другом. [3]

UEFI (от англ. Unified Extensible Firmware Interface, единый расширяемый микропрограммный интерфейс) был разработан консорциумом UEFI Forum на основе спецификаций, изначально предоставленных корпорацией Intel как преемника BIOS (хотя большинство образов микропрограммного обеспечения UEFI предоставляют поддержку для устаревших сервисов BIOS).

[3] Вонг А. Справочник по параметрам BIOS. — 2005

UEFI обеспечивает поддержку современного оборудования, модульной архитектуры, не зависящей от архитектуры ЦПУ и драйверов, загрузочной сети и безопасной загрузки. [4]

Важным преимуществом предлагаемого решения является возможность вывода логов от гостевой операционной системы вместе с диагностическими данными от самого эмулятора. В результате логи драйверов и сервисов, работающих в гостевых операционной системах, и сообщения эмулятора печатаются в одном месте в хронологическом порядке. Это позволяет видеть полную картину и более эффективно выявлять и исправлять ошибки. Техподдержке и разработчикам не нужно собирать диагностическую информацию из разных источников и пытаться сопоставить события по времени, ведь в одном файле доступны все логи, упорядоченные по времени.

Для решения проблем, описанных выше, предлагается механизм, основанный на использовании общей памяти. На стороне эмулятора реализуется виртуальное устройство, служащее интерфейсом для хранения метаинформации о буфере. В операционной системе разработан драйвер, обеспечивающей взаимодействие с данным устройством и передачу диагностических данных.

Объект исследования в данной работе — высокопроизводительный механизм передачи информации между гостевой операционной системой и эмулятором с минимальными временными задержками.

Предмет исследования — многопоточный кольцевой буфер без блокировок как средство передачи диагностической информации в среде виртуализации.

Целью дипломной работы является разработка комплекса программных компонентов для организации высокоскоростной передачи диагностических информации в среде виртуализации. В состав разрабатываемого решения входят: виртуальное устройства для виртуальной машины Stratovirt [5]; драйвер для среды UEFI, позволяющий передавать диагностические сообщения на самых ранних этапах работы эмулятора; а также драйвер для

[4] Энциклопедия «Касперского» — UEFI

[5] https://www.openeuler.org/en/other/projects/stratovirt/

операционной системы Windows, обеспечивающий возможность передачи диагностической информации от приложений, сервисов и других драйверов, работающих внутри гостевой системы.

Таким образом, постановка задачи заключается в разработке данных трёх компонентов. Для достижения поставленной цели требуется выполнить следующие этапы:

1. Изучить устройство и архитектуру эмулятора stratovirt.
2. Разработать виртуальное устройство на стороне эмулятора.
3. Изучение устройства и архитектуру UEFI.
4. Разработать драйвер для UEFI.
5. Разобраться в коде некоторых драйверов для ОС Windows и изучить API, предоставляемые операционной системой для разработки драйверов.
6. Реализовать драйвер для ОС Windows.
7. Сравнить реализованный канал для передачи данных с другими существующими.

Методологическую основу работы составляют: анализ технической документации и литературы о разработке драйверов для UEFI и Windows, материалы по lock-free алгоритмам и методы отладки, применяемые в среде виртуализации.

Постановка задачи определяет структуру дипломной работы, которая включает введение, четыре главы и заключение.

Введение задаёт цели и задачи работы: в этой части определяются объект и предмет исследования и раскрывается значимость.

В первой главе рассматриваются устройство и архитектура виртуальной машины stratovirt.

Вторая глава посвящена архитектуре UEFI и принципам разработки драйверов В среде UEFI.

В третьей главе описываются подходы к разработке драйверов в операционной системе Windows.

Четвёртая глава посвящена описанию реализации механизма передачи диагностических данных.

Заключение содержит обобщение результатов проведённого исследования, сравнение с другими механизмами, а также описание возможных сценариев использования разработанного решения.

2 Архитектура виртуальной машины stratovirt

3 Драйверы в среде UEFI

4 Драйверы в операционной системе Windows

5 Реализация механизма сбора диагностической информации

Разработанный механизм сбора диагностической информации включает три основных компонента:

1. Виртуальное устройства на стороне гипервизора Stratovirt.
2. Драйвер для среды UEFI.
3. Драйвер для операционной системы Windows.

Каналом взаимодействия между данными компонентами служит кольцевой буфер, расположенный в общей памяти, разделяемой между гостевой ОС и гипервизором.

Виртуальное устройство эмулируется в гипервизоре Stratovirt и выступает в роли единой точки сбора диагностической информации. Таким образом, основными функциями устройства являются:

1. Приём диагностических сообщений от драйверов гостевых систем.
2. Сериализация сообщений и запись их в файл.
3. Хранение метаинформации о буфере: его размере и типе поведения при переполнении.
4. Предоставление доступа к метаинформации посредством регистров виртуального устройства.

Драйвер для среды UEFI выполняет следующие задачи:

1. Выделение буфера в гостевой памяти типа runtime: доступной после окончания работы UEFI (после вызова ExitBootServices) и во время работы операционной системы.
2. Предоставление интерфейса для записи диагностических сообщения в буфер.
3. Регистрация обработчиков событий ExitBootServices и SetVirtualAddressMap для обеспечения работы драйвера с EfiRuntimeService.

Для упрощения интеграции с существующими UEFI модулями была разработана собственная версия DebugLib, переопределяющая стандартный функции логирования. Эти функции обращаются к вышеупомянутому драйверу для передачи сообщений.

Для обеспечения взаимодействия компонентов гостевой системы с буфером реализован драйвер, позволяющий:

1. User-space приложениям и сервисам взаимодействовать с буфером.
2. Kernel-space драйверам взаимодействовать с буфером.

Общая архитектура механизма представлена на рисунке 1.

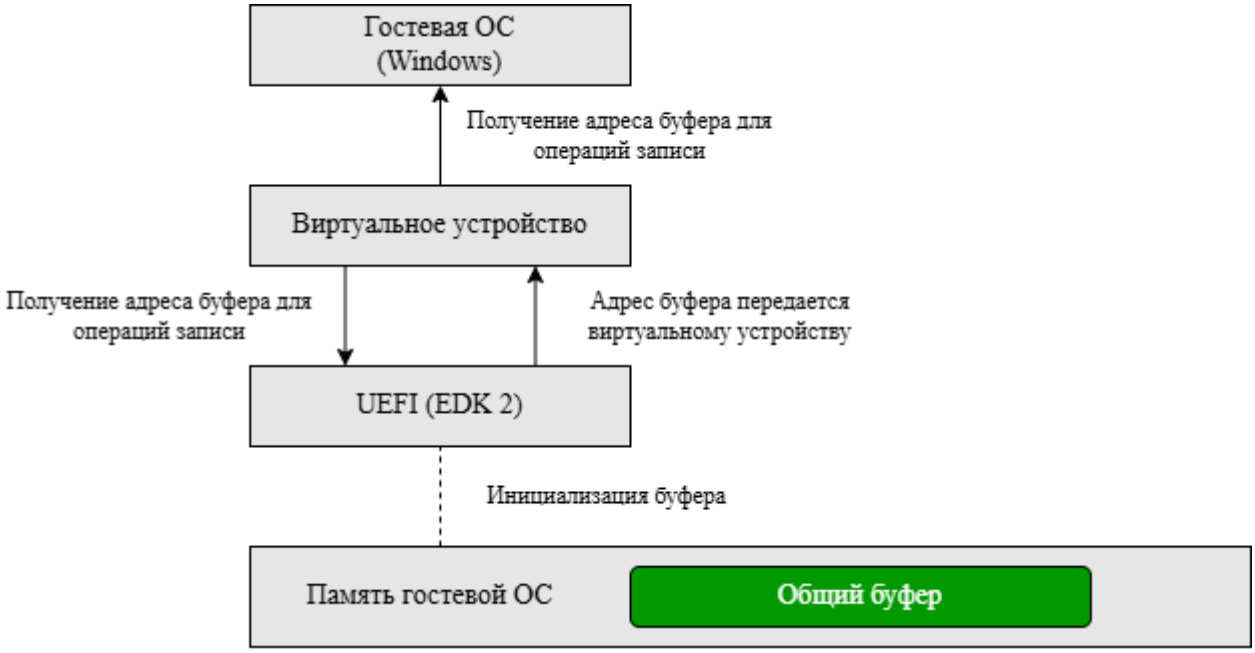

Рис. 1: Архитектура механизма обмена диагностической информацией

6 Заключение

Здесь надо перечислить все результаты, полученные в ходе работы. Из текста должно быть понятно, в какой мере решена поставленная задача.

Список литературы

[1] *Mott-Smith, H.* The theory of collectors in gaseous discharges / H. Mott-Smith, I. Langmuir // *Phys. Rev.* — 1926. — Vol. 28.

[2] *Морз, Р.* Бесстолкновительный PIC-метод / Р. Морз // Вычислительные методы в физике плазмы / Ed. by Б. Олдера, С. Фернбаха, М. Ротенберга. — М.: Мир, 1974.

[3] *Киселёв, А. А.* Численное моделирование захвата ионов бесстолкновительной плазмы электрическим полем поглощающей сферы / А. А. Киселёв, Долгоносов М. С., Красовский В. Л. // Девятая ежегодная конференция «Физика плазмы в Солнечной системе». — 2014.